Understanding Practical 1

Location-Based Messaging App & Ubiquitous Computing

Part 1 — Why This Practical Exists

1.1 What is Ubiquitous Computing?

Ubiquitous computing (a term coined by Mark Weiser) means computing that is:

- **Embedded —** built into everyday objects/devices, not a separate "computer" you sit at.
- **Context-aware —** the system senses something about the environment (location, time, motion, light, proximity) and reacts to it automatically.
- **Invisible / seamless —** the user doesn't manage the technology; the technology adapts to the user.
- **Always available —** carried with the person (a phone is the best modern example).

> **The one-line idea**
> Your phone already knows where you are. Ubiquitous computing is about writing software that uses that awareness to change its behavior automatically — without you having to type in your location.

1.2 How does this practical prove that concept?

The aim of the practical is: Design and Develop a Location-Based Messaging App. Here is the causal chain that connects the app to the concept:

1. The phone has a GPS/location sensor embedded in it (ubiquitous hardware).
2. The app senses the phone's current location (context sensing) — this is the "ubiquitous" part.
3. The app reasons about that context: "Is this location inside Mumbai's coordinate range or not?"
4. The app reacts by displaying a different message depending on the answer — without the user typing anything.

This is a textbook example of a context-aware application: sense → interpret → adapt. Use this as the justification in your practical's conclusion:

> **Conclusion line to remember**
> "The app demonstrates ubiquitous computing because it senses ambient context (GPS location) transparently and adapts its output (the message shown) accordingly, fulfilling the core UbiComp principle of context-aware, invisible computing."

Part 2 — Android Studio: The Absolute Basics

2.1 What is Android Studio?

It's an IDE (Integrated Development Environment) — a text editor + compiler + emulator manager + debugger, all in one, built specifically for making Android apps. If you're used to writing core Java in a plain text editor and compiling separately, Android Studio automates all of that, and also lets you simulate a phone on your PC.

2.2 What is Gradle?

Gradle is the build system. Plain Java can be compiled with a single command like javac Hello.java. Android apps are far more complex — dozens of files, libraries, resources, permissions — so Gradle is the tool that:

- Downloads any external libraries your code needs (like the Google Location library used here).
- Compiles all your Java files.
- Packages everything (code + images + XML layouts) into one installable file called an APK.

Every time you click “Run”, Gradle runs a “build” behind the scenes before the app installs on the emulator/phone.

2.3 What is an Emulator (AVD)?

Since not everyone has an Android phone handy (or wants to plug one in via USB), Android Studio can simulate a virtual Android phone on the PC screen. That's what creating a device (e.g. “Pixel 8”) in Device Manager does — it creates a virtual phone that runs on your laptop, complete with a simulated GPS you can manually move around.

2.4 What is an “Activity”?

An Activity = one screen of the app. Simple apps might have just one Activity (MainActivity) — this practical only needs one screen: a button and some text.

Why “Empty Views Activity” and not “Empty Activity”? This is a doubt every student raises:

- **Empty Activity** (newer template) generates a screen using Jetpack Compose — a completely different, code-only way of building UI (no XML layout file at all).
- **Empty Views Activity** (the classic template) generates the traditional View-based UI system: a Java file paired with an XML layout file (activity_main.xml).

Since this practical teaches the classic View + XML approach in Java, “Empty Views Activity” is mandatory — it's the only template that gives you the file structure the code expects.

Part 3 — Project Folder Structure: What Each File Does

When you open the project, you'll see a tree roughly like this:

```
LocationMessagingApp/                  the whole project (one Gradle "project")
└─ app/                                 the actual application module
   ├─ build.gradle.kts                  "recipe" for building THIS app module
   └─ src/main/
      ├─ java/.../MainActivity.java      YOUR LOGIC lives here
      └─ res/                             "resources" — non-code assets
         ├─ layout/activity_main.xml      YOUR UI / SCREEN DESIGN lives here
         ├─ values/strings.xml            all text shown in the app, centralized
         └─ mipmap/...                     app icons
├─ AndroidManifest.xml                  the app's "identity card" + permissions
├─ gradle/libs.versions.toml            central list of library VERSION NUMBERS
└─ build.gradle.kts  (project-level)    settings shared by the WHOLE project
```

Why does Android split code (Java) from UI (XML) into two files?

This is the single most important structural concept to understand — and the most common source of confusion.

Separation of concerns: activity_main.xml describes what the screen looks like (buttons, text positions, sizes, colors) — purely declarative, no logic. MainActivity.java describes what happens (what the button does when clicked, how to read GPS, what text to change). This split means a designer could redesign the screen without touching a single line of Java, and a developer can change the logic without breaking the layout. It mirrors HTML/CSS vs JavaScript, if you know web development — same philosophy.

Why a "project-level" build.gradle.kts AND an "app-level" one?

- **Project-level file** applies to the entire project — think of it as the outer wrapper. In a real company app, one project might contain multiple app modules (e.g., a phone app + a wearable app); the project-level file holds settings shared across all of them.
- **App-level file** configures just this one app: its package name, minimum Android version supported, and — critically — its dependencies (external libraries).

Why does libs.versions.toml exist?

Older Android projects hardcoded version numbers directly inside build files (e.g. implementation("androidx.appcompat:appcompat:1.7.1") scattered everywhere). Newer Android Studio projects use this "version catalog" system to centralize every version number in one file, so if ten different files use the same library, you only update the version number in one place. libs.versions.toml is a lookup table of name → version and name → library coordinates, which build.gradle.kts then references using entries like libs.appcompat.

What is AndroidManifest.xml and why must it exist?

Think of it as the app's passport / ID card, read by the Android OS before anything else runs. It answers three questions the OS needs upfront:

1. What is this app called, and what does its icon look like?
2. What screens (Activities) does it have, and which one launches first?
3. What permissions (sensitive capabilities) does it need to ask the user for?

The two critical lines for this practical are:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

Without declaring these here, Android will refuse to let your app touch GPS at all — no matter what your Java code says. This is Android's core security model: anything privacy-sensitive (location, camera, contacts, microphone) must be declared in the Manifest, and the user must approve it at runtime.

What is strings.xml, and why not just type text directly in Java/XML?

strings.xml centralizes every piece of user-visible text. Two big reasons:

1. **Localization —** if you wanted a Hindi version of the app, you'd just add a values-hi/strings.xml with translated text; zero Java/XML code changes.
2. **Maintainability —** if the welcome message needs to change, you edit one line in one file instead of hunting through Java code.

Notice this formatting trick used in strings.xml:

```
<string name="latitude">Latitude : %1$.6f</string>
```

%1$ is a format placeholder meaning “first argument passed in,” and .6f means “display as a decimal number with 6 digits after the decimal point.” GPS coordinates need that many decimal places to be precise — each extra digit of precision is roughly a 10x improvement in accuracy, down to about 11 cm at 6 decimals. This is filled in later by the Java code via getString(R.string.latitude, latitude).

Part 4 — activity_main.xml, Line by Line

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">
```

- `LinearLayout` — a container that arranges its children in one line (like a stack). This is the simplest layout type to start with, versus ConstraintLayout, which is more powerful but has a steeper learning curve.
- `xmlns:android="..."` — an XML namespace declaration. It tells the XML parser “any attribute prefixed with android: should be interpreted using Android's own attribute dictionary.” Every Android layout file needs this line once, at the root tag — treat it as boilerplate that must be there.
- `layout_width / layout_height = "match_parent"` — this container should stretch to fill its parent (the whole screen).
- `orientation="vertical"` — stack children top-to-bottom (as opposed to horizontal, side-by-side).
- `gravity="center"` — center all child views horizontally within this layout.
- `padding="20dp"` — leave a 20 dp margin of empty space around the inside edges. dp = “density-independent pixel” — a unit that looks the same physical size across different screen resolutions (unlike a raw pixel, which would look tiny on a high-res screen and huge on a low-res one).

```
<TextView android:id="@+id/txtTitle" ... android:text="Location Based Messaging App" />
```

- `TextView` — a UI widget that just displays text (non-editable). Think of it as a <p> or in HTML terms.
- `android:id="@+id/txtTitle"` — assigns this widget a unique identifier, txtTitle. The “+” tells Android Studio: “create a new ID resource called txtTitle and remember it.” This ID is how the Java code will later find and control this exact widget (findViewById(R.id.txtTitle)).
- The rest are cosmetic attributes: text size, bold style, bottom margin.

```
<Button android:id="@+id/btnLocation" ... android:text="Get Current Location"/>

<TextView android:id="@+id/txtLatitude" .../>
<TextView android:id="@+id/txtLongitude" .../>
<TextView android:id="@+id/txtMessage" .../>
```

Button is a clickable widget; its ID is what the Java code hooks a click-listener onto. The three more TextViews initially show placeholder text (“Latitude :”, “Longitude :”, “Location Message”). These three IDs (txtLatitude, txtLongitude, txtMessage) are exactly the three fields the Java code will update once GPS data comes back.

> **Key mental model**
> Every widget with an id in this XML file has a matching findViewById(R.id....) call in MainActivity.java. This XML-to-Java linkage (via the auto-generated R class) is the single most important concept for View-based Android UI.

Part 5 — MainActivity.java, Line by Line

Package and imports

```
package com.example.locationmessagingapp;
```

Declares which package (namespace) this class belongs to — must match the applicationId / namespace set in build.gradle.kts and the Manifest. Purely organizational, same as any Java package.

```
import android.Manifest;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.location.Priority;
```

There are two groups here:

- `android.*` and `androidx.*` — Android's own built-in framework classes (permissions handling, UI widgets, the base Activity class).
- `com.google.android.gms.location.*` — this is Google Play Services' Location library — not part of core Android, an external dependency that had to be added in build.gradle.kts (play-services-location). This is the modern, battery-efficient, recommended way to get GPS location.

Class declaration and fields

```
public class MainActivity extends AppCompatActivity {
```

MainActivity extends AppCompatActivity, meaning it inherits all the built-in behavior of "being a screen" (handling the Android lifecycle, back button, theming, etc.) from the Android framework. Every Activity in a Views-based app must extend AppCompatActivity (or a subclass of it).

```
private TextView txtLatitude, txtLongitude, txtMessage;
private FusedLocationProviderClient fusedLocationClient;
private static final int LOCATION_PERMISSION_REQUEST = 100;
```

- Three TextView reference variables — Java-side handles that will point to the XML widgets.
- `fusedLocationClient` — the object that actually knows how to ask the phone's GPS hardware for a location.
- `LOCATION_PERMISSION_REQUEST = 100` — an arbitrary integer code used to identify which permission request is being answered when Android calls back later (useful when an app requests multiple different permissions). The number 100 has no special meaning; it's just a label.

onCreate() — the entry point

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
```

onCreate() is a lifecycle method — Android automatically calls this the moment the screen is created. Every Activity must override it. savedInstanceState carries saved screen data if the Activity was previously destroyed and recreated (e.g., after a screen rotation); it's not deeply used here, but must be passed to super or the app crashes.

setContentView(R.layout.activity_main) is the line that connects the XML file to this Java class. R is an auto-generated class (built by Android Studio at compile time) containing an ID for every resource — layouts, strings, ids, images. This tells Android: “render this screen using that XML blueprint.”

```
Button btnLocation = findViewById(R.id.btnLocation);
txtLatitude = findViewById(R.id.txtLatitude);
txtLongitude = findViewById(R.id.txtLongitude);
txtMessage = findViewById(R.id.txtMessage);

fusedLocationClient = LocationServices.getFusedLocationProviderClient(this);

btnLocation.setOnClickListener(v -> getCurrentLocation());
```

findViewById(R.id.xxx) walks the rendered XML layout tree and returns the actual Java object representing that widget, using the id set in the XML — this is the XML–Java link discussed in Part 4. The FusedLocationProviderClient is then initialized, tied to this Activity.

The click listener attaches a click handler to the button. v -> getCurrentLocation() is a lambda expression (shorthand for an anonymous inner class implementing View.OnClickListener). In plain words: “When this button is clicked, run the getCurrentLocation() method.” The longhand equivalent is:

```
btnLocation.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) { getCurrentLocation(); }
});
```

getCurrentLocation() — the core logic

```
private void getCurrentLocation() {
    if (ActivityCompat.checkSelfPermission(
            this,
            Manifest.permission.ACCESS_FINE_LOCATION)
        != PackageManager.PERMISSION_GRANTED) {
```

This checks: has the user already granted the location permission? Android requires this check every single time you're about to use a sensitive permission — you cannot assume it was granted once and never check again, since the user can revoke permissions at any time from Settings.

```
        ActivityCompat.requestPermissions(
            this,
            new String[]{
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION
            },
            LOCATION_PERMISSION_REQUEST
```

```
        );
        return;
    }
```

If not granted yet, this pops up the system permission dialog asking the user to Allow/Deny. LOCATION_PERMISSION_REQUEST tags this specific request so it's recognized in the callback. Importantly, the method returns here — execution cannot continue linearly, because the user's response comes back asynchronously.

```
    fusedLocationClient.getCurrentLocation(
        Priority.PRIORITY_HIGH_ACCURACY,
        null
    ).addOnSuccessListener(this, location -> {
```

If permission was already granted, this actually requests the location. Priority.PRIORITY_HIGH_ACCURACY asks for the most precise GPS fix possible (uses more battery, but better for demonstrating exact coordinates). addOnSuccessListener(this, location -> {...}) is an asynchronous callback: fetching GPS isn't instant, so instead of blocking the app while waiting, a lambda is registered that runs once the location becomes available.

```
        if (location != null) {
            double latitude = location.getLatitude();
            double longitude = location.getLongitude();

            txtLatitude.setText(getString(R.string.latitude, latitude));
            txtLongitude.setText(getString(R.string.longitude, longitude));
```

This extracts latitude/longitude as doubles. getString(R.string.latitude, latitude) fetches the string template from strings.xml (“Latitude : %1$.6f”) and substitutes in the actual number — this is where the placeholder from Part 3 gets filled.

```
            if (latitude >= 18.8 && latitude <= 19.5 &&
                longitude >= 72.7 && longitude <= 73.2) {
                txtMessage.setText(R.string.welcome_message);
            } else {
                txtMessage.setText(R.string.outside_message);
            }
```

This is the payoff of the whole practical

The coordinate range (18.8–19.5° N, 72.7–73.2° E) is roughly the bounding box of the Mumbai metropolitan region. If the sensed GPS location falls inside that box, the app shows “Welcome to Mumbai”; otherwise it shows “You are outside Mumbai.” This is literally: sense context → branch logic on context → adapt output — the definition of context-aware computing, executed in four lines of code.

```
        } else {
            txtMessage.setText(R.string.unable_location);
        }
    });
}
```

Defensive handling: sometimes getCurrentLocation() succeeds but returns null (e.g., GPS not yet warmed up, or location services off) — this shows a friendly fallback message instead of crashing.

onRequestPermissionsResult() — handling the user's permission decision

```
@Override
public void onRequestPermissionsResult(
        int requestCode,
        @NonNull String[] permissions,
        @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == LOCATION_PERMISSION_REQUEST) {
        if (grantResults.length > 0 &&
            grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            getCurrentLocation();
        } else {
            txtMessage.setText(R.string.permission_denied);
        }
    }
}
```

Android automatically calls this method after the user taps Allow/Deny on the permission popup. requestCode lets us confirm this callback is about our request (matches LOCATION_PERMISSION_REQUEST = 100 — useful in bigger apps with multiple permission requests happening). If granted, getCurrentLocation() is called again — this time the permission check at the top will pass, and it proceeds to actually fetch GPS. If denied, a message is shown rather than silently failing.

Part 6 — The Complete End-to-End Flow

This sequence ties every file together, from launch to the final message on screen:

1. App launches → Android reads AndroidManifest.xml, sees MainActivity is the launcher activity, and starts it.
2. onCreate() runs → setContentView(R.layout.activity_main) inflates the XML layout onto the screen. Title, button, and three placeholder TextViews appear.
3. findViewById() calls wire up Java-side handles to each XML widget.
4. User taps “Get Current Location” → the lambda click-listener fires → getCurrentLocation() runs.
5. Permission check — first time, permission is NOT granted → system dialog pops up asking “Allow LocationMessagingApp to access this device's location?”
6. User taps Allow → Android calls onRequestPermissionsResult() → sees permission granted → calls getCurrentLocation() again.
7. Permission check passes this time → fusedLocationClient.getCurrentLocation() asks the phone's (or emulator's simulated) GPS hardware for coordinates.
8. GPS returns a location (asynchronously) → the success-listener lambda runs → latitude/longitude are extracted and displayed.
9. Coordinate check → if inside the Mumbai bounding box, “Welcome to Mumbai” is shown; otherwise “You are outside Mumbai.”
10. Result on screen: Title → Button → Latitude → Longitude → context-aware Message, all updated live from real sensor data.

> **For your conclusion**

> This entire chain — sensing → asynchronous system callback → permission negotiation → context branching → adaptive UI — is exactly what you write up as proof that ubiquitous computing principles were implemented.

Part 7 — Frequently Asked Questions

Q: Why do we need to ask permission at all? Why can't the app just read GPS directly?

A: Location is one of Android's "dangerous" permission categories (along with camera, microphone, contacts, SMS) because it's privacy-sensitive. Since Android 6.0 (API 23), these must be requested at runtime, not just declared in the Manifest — the Manifest declaration is necessary but not sufficient.

Q: Why is getCurrentLocation() called twice in the flow (once from the button, once from the permission callback)?

A: The first call's job is really just "check permission and, if missing, ask for it" — it returns immediately without getting any location. Only the second call (triggered after the user grants permission) actually proceeds past the permission check to fetch GPS. It's the same method reused for both jobs, which avoids duplicating the permission-check logic.

Q: What happens if the emulator doesn't have a "real" location?

A: Emulators simulate GPS. In the running emulator's "Extended Controls" panel (three-dot menu on the toolbar) → Location, you can manually set a latitude/longitude, or even simulate a route. Set it to a Mumbai coordinate (e.g., 19.07, 72.87) and click Send to test the "Welcome" branch; set it outside that range to test the other branch.

Q: Why double for latitude/longitude and not float or int?

A: GPS coordinates require high decimal precision (a change in the 5th–6th decimal place is a few meters on the ground). double gives about 15–17 significant digits, more than enough; int obviously can't hold decimals at all, and float has less precision than we'd want for repeated calculations.

Q: What's the difference between ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION?

A: FINE = GPS-based, most accurate (a few meters). COARSE = network/cell-tower/Wi-Fi-based, less accurate (hundreds of meters to a few km), but uses less battery. Requesting both gives Android the flexibility to satisfy the fine-grained request but fall back gracefully.

Q: Why is the Mumbai boundary written as four separate comparisons instead of some location library function?

A: This is deliberately simple for teaching purposes — a bounding-box check is the easiest way to demonstrate "is my current context near a place of interest" without adding complexity like reverse-geocoding or the Google Maps API. In a production app, you'd typically use a proper geofencing API for this instead of hardcoded coordinates.

Q: Why does the app need internet/Play Services if GPS is a hardware sensor?

A: FusedLocationProviderClient (part of Google Play Services) combines multiple signals (GPS, Wi-Fi, cell towers, sensors) to produce a more accurate, battery-efficient location fix than raw GPS alone — hence "fused." This is why the play-services-location dependency had to be added in build.gradle.kts, and why an emulator needs a "Google Play" system image (not a plain AOSP one) to run this app.

Q: What if I run this on my physical phone instead of the emulator?

A: Same code, same permissions dialog — except now real GPS coordinates from wherever you physically are will show up. Walk outside the Mumbai boundary (or set a VPN/mock location app) and watch the message change live.

Q: Why AppCompatActivity instead of just Activity?

A: AppCompatActivity is a backward-compatible version that brings newer Android UI features (like modern toolbars/themes) to devices running older Android versions. It's the standard base class recommended for all View-based apps today; plain Activity is the more "raw," rarely-used base class.

Q: I changed a string name in strings.xml but the Java code broke — why?

A: If you rename a <string name="..."> key without also updating every reference in Java, the project won't compile — the R class is auto-generated from strings.xml, so Java and XML resource names must always match exactly.

Part 8 — Quick Recap

1. Ubiquitous computing means your device senses the environment and reacts, without you telling it what to do.
2. This app senses your GPS location and shows a different message depending on whether you're in Mumbai — that's the whole "ubiquitous" part.
3. Android splits every screen into two paired files: XML for what it looks like, Java for what it does. IDs in the XML are how the Java code finds and controls each widget.
4. Location is a sensitive permission — Android forces apps to explicitly ask for it at runtime, and the code has to handle both "granted" and "denied" outcomes gracefully.
5. Once coordinates are available, the app does a simple bounding-box check — this is the "sense → decide → adapt" loop that defines context-aware computing.
6. Everything else — Gradle files, the manifest, the version catalog — exists to configure permissions, dependencies, and build settings so that this small piece of logic can actually run on a real device.